

Giới thiệu đồ thị khoảng, đồ thị dây cung và đồ thị hoàn hảo

Dzx

Nguồn gốc: Introduction to interval graphs, chordal graphs and perfect graphs

Graph Theory / Interval graphs, chordal graphs and perfect graphs

1 Nội dung

Bài này giới thiệu ba lớp đồ thị thường gặp trong mô hình hóa và thuật toán: **đồ thị khoảng** (interval graph), **đồ thị dây cung** (chordal graph) và **đồ thị hoàn hảo** (perfect graph). Trọng tâm là cách nhìn từ biểu diễn hình học của các khoảng sang các tính chất tổ hợp: tô màu tối ưu, clique cực đại, thứ tự khử hoàn hảo, nhận biết đồ thị dây cung bằng LexBFS hoặc MCS, và nhận biết đồ thị khoảng bằng thứ tự liên tiếp của các clique cực đại.

Các hình trong slide gốc chủ yếu minh họa các khoảng trên một trục, các cạnh giao nhau giữa chúng, và các mẫu biến đổi của PQ-tree. Ở bản ghi chú này các hình đó được diễn giải bằng lời thay vì chèn lại ảnh.

2 Đồ thị khoảng và bài POJ 1083

2.1 Mô hình hóa bài Moving Tables

Trong bài *Moving Tables* (POJ 1083), một công ty có 400 phòng nằm hai bên một hành lang hẹp. Các phòng lẻ nằm ở một phía, các phòng chẵn nằm ở phía còn lại; hai phòng đối diện nhau dùng chung cùng một đoạn hành lang. Cần chuyển một số bàn từ phòng này sang phòng khác. Nếu hai lần chuyển bàn phải đi qua cùng một đoạn hành lang thì chúng không thể diễn ra đồng thời. Mỗi bàn mất 10 phút để chuyển, hỏi thời gian ít nhất để hoàn thành tất cả các lần chuyển.

Với mỗi lần chuyển từ phòng a sang phòng b , ta đổi số phòng về chỉ số đoạn hành lang

$$p(x) = \left\lceil \frac{x}{2} \right\rceil.$$

Lần chuyển đó chiếm các đoạn từ $\min(p(a), p(b))$ tới $\max(p(a), p(b))$, tức một khoảng trên trục hành lang. Hai lần chuyển xung đột đúng khi hai khoảng tương ứng giao nhau. Vì vậy ta dựng một đồ thị: mỗi lần chuyển là một đỉnh, hai đỉnh nối cạnh nếu hai khoảng của chúng giao nhau. Cần chia các đỉnh thành ít nhất số nhóm sao cho trong mỗi nhóm không có cạnh; đó chính là bài toán tô màu đồ thị.

Với đồ thị tổng quát, tìm số màu nhỏ nhất là NP-khó. Nhưng đồ thị vừa dựng là đồ thị khoảng, nên số màu nhỏ nhất bằng kích thước clique lớn nhất, tức bằng số khoảng lớn nhất cùng phủ một đoạn hành lang. Đáp án của bài là

$$10 \cdot \max_x \# \{\text{lần chuyển đi qua đoạn } x\}.$$

2.2 Định nghĩa

Một **khoảng** là một đoạn trên trục số thực có hai đầu mút; đầu mút có thể mở hoặc đóng. Cho một họ khoảng $\{I_v\}$, ta định nghĩa **đồ thị giao** của họ này như sau: mỗi khoảng I_v ứng với một đỉnh v , và giữa hai đỉnh v, w có cạnh khi và chỉ khi

$$I_v \cap I_w \neq \emptyset.$$

Một đồ thị G được gọi là **đồ thị khoảng** nếu nó là đồ thị giao của một họ khoảng nào đó.

Các slide gốc đưa ra hai hình đối chiếu: một họ khoảng có thể biểu diễn đúng một đồ thị khoảng, và một đồ thị khác không thể biểu diễn bằng các khoảng trên cùng một trục. Điểm quan trọng là quan hệ kề phải đến từ giao nhau trên một đường thẳng, không phải từ một cách vẽ tùy ý của đồ thị.

2.3 Các dạng đầu mút và thứ tự tự nhiên

Định lý. Các đồ thị sinh bởi khoảng mở, khoảng đóng và khoảng nửa mở nửa đóng là cùng một lớp đồ thị.

Ý tưởng chứng minh. Vì các khoảng nằm trên trục số thực liên tục, ta có thể dịch hoặc co giãn các đầu mút một lượng rất nhỏ để loại bỏ trường hợp giao nhau chỉ tại đầu mút, mà không làm thay đổi các quan hệ giao nhau cần giữ. Do đó kiểu mở hay đóng của đầu mút không làm thay đổi lớp đồ thị thu được.

Hệ quả. Mọi đồ thị khoảng đều có một biểu diễn bằng khoảng trong đó không có hai đầu mút nào trùng nhau. Khi đó ta có thể sắp xếp các đỉnh theo thứ tự tăng dần của đầu mút trái của khoảng đại diện. Thứ tự này được gọi là **thứ tự tự nhiên** của đồ thị khoảng.

Ký hiệu thứ tự tự nhiên là

$$v_1, v_2, \dots, v_n.$$

Với một đỉnh v_i , đặt

$$\text{Pred}(v_i) = \{v_j \mid (v_i, v_j) \in E, j < i\}.$$

Ta gọi đây là tập các **tiền nhiệm** của v_i trong thứ tự đang xét.

Tính chất then chốt. Với thứ tự tự nhiên của một đồ thị khoảng,

$$\{v_i\} \cup \text{Pred}(v_i)$$

là một clique. Thật vậy, mọi tiền nhiệm của v_i có đầu mút trái nằm trước đầu mút trái của v_i nhưng vẫn giao với I_{v_i} , nên chúng đều chứa điểm đầu trái của I_{v_i} . Do đó chúng đôi một giao nhau.

3 Tô màu và clique lớn nhất trên đồ thị khoảng

Từ thứ tự tự nhiên $v_1, \dots, v_n$, ta tô màu tham lam:

1. Duyệt các đỉnh theo thứ tự $v_1, v_2, \dots, v_n$.
2. Khi xét v_i , gán cho nó màu nhỏ nhất chưa xuất hiện trong $\text{Pred}(v_i)$.

Thuật toán này cho một tô màu tối ưu của đồ thị khoảng. Lý do là tại thời điểm xét v_i , tập $\{v_i\} \cup \text{Pred}(v_i)$ là một clique, nên nếu $\text{Pred}(v_i)$ đã dùng k màu khác nhau thì mọi cách tô màu đều cần ít nhất $k + 1$ màu

cho clique đó. Thuật toán tham lam dùng đúng số màu cần thiết ở từng bước, nên tổng số màu nó dùng bằng kích thước clique lớn nhất:

$$\chi(G) = \omega(G).$$

Ở đây $\chi(G)$ là số màu nhỏ nhất, còn $\omega(G)$ là kích thước clique lớn nhất của G .

Với bài POJ 1083, clique lớn nhất chính là số lần chuyển cùng đi qua một đoạn hành lang. Vì vậy không cần giải bài tô màu tổng quát; chỉ cần đếm độ phủ lớn nhất của các khoảng.

4 Thứ tự khử hoàn hảo

Thứ tự khử hoàn hảo (perfect elimination order, PEO) là một thứ tự $v_1, \dots, v_n$ sao cho với mọi i , tập $\text{Pred}(v_i)$ là một clique. Theo tính chất ở trên, thứ tự tự nhiên của mọi đồ thị khoảng là một thứ tự khử hoàn hảo.

PEO là công cụ trung tâm để xử lý các bài toán tối ưu trên đồ thị dây cung. Khi đã có PEO, nhiều bài toán vốn khó trên đồ thị tổng quát trở nên đơn giản hơn, chẳng hạn:

- tìm clique lớn nhất;
- tô màu tối ưu;
- tìm tập độc lập lớn nhất;
- phủ đỉnh nhỏ nhất trong một số lớp bài toán liên quan;
- phủ bằng số clique ít nhất.

Với quy ước PEO dùng tập tiền nhiệm như trên, tô màu tham lam theo thứ tự PEO cũng tối ưu trên đồ thị dây cung. Nếu dùng quy ước phổ biến khác là “các láng giềng xuất hiện sau tạo thành clique”, thì thứ tự đó chính là thứ tự ngược lại của quy ước trong bài này.

5 Đồ thị dây cung

5.1 Định nghĩa

Một đồ thị được gọi là **đồ thị dây cung** nếu nó không có chu trình cảm sinh độ dài ít nhất 4. Nói tương đương, mọi chu trình đơn độ dài ít nhất 4 đều có một **dây cung**, tức một cạnh nối hai đỉnh không liên tiếp trên chu trình.

Mọi đồ thị khoảng đều là đồ thị dây cung, vì thứ tự tự nhiên của nó là PEO, và như định lý dưới đây cho thấy, có PEO là đúng đặc trưng của đồ thị dây cung.

5.2 Liên hệ với thứ tự khử hoàn hảo

Định lý. Nếu một đồ thị G có thứ tự khử hoàn hảo thì G là đồ thị dây cung.

Chứng minh phản chứng. Giả sử tồn tại một chu trình cảm sinh C độ dài ít nhất 4. Lấy đỉnh v_i có chỉ số lớn nhất trên chu trình. Hai láng giềng của v_i trên C đều nằm trước v_i , nên theo PEO chúng phải kề nhau. Khi đó ta có một dây cung của C , mâu thuẫn với việc C là chu trình cảm sinh.

Định lý. Một đồ thị G là đồ thị dây cung khi và chỉ khi G có thứ tự khử hoàn hảo.

Chiều còn lại dựa trên khái niệm điểm đơn thuần. Một đỉnh v được gọi là **điểm đơn thuần** nếu mọi láng giềng của v tạo thành một clique.

Bổ đề. Mọi đồ thị dây cung đều có ít nhất một điểm đơn thuần. Nếu đồ thị dây cung đó không phải đồ thị đầy đủ, nó có ít nhất hai điểm đơn thuần không kề nhau.

Bổ đề. Mọi đồ thị con cảm sinh của một đồ thị dây cung vẫn là đồ thị dây cung.

Từ hai bổ đề này, ta xây dựng PEO bằng cách lặp lại thao tác chọn một điểm đơn thuần, đặt nó vào cuối thứ tự khử theo quy ước “láng giềng phía sau tạo thành clique”, rồi xóa nó khỏi đồ thị. Đảo ngược thứ tự thu được sẽ phù hợp với quy ước Pred của bài này.

6 Nhận biết đồ thị dây cung

Có hai thuật toán tuyến tính thường dùng để sinh một thứ tự ứng viên cho PEO: **MCS** (maximum cardinality search) và **LexBFS** (lexicographical breadth-first search). Sau đó ta kiểm tra thứ tự này trong $O(n + m)$.

6.1 Maximum cardinality search

MCS duy trì cho mỗi đỉnh chưa chọn một trọng số, ban đầu bằng 0. Lặp lại n lần:

1. chọn một đỉnh chưa chọn có trọng số lớn nhất;
2. đưa nó vào thứ tự;
3. tăng trọng số của mọi láng giềng chưa chọn của nó lên 1.

Trọng số của một đỉnh chính là số láng giềng đã được chọn trước đó. Với đồ thị dây cung, thứ tự do MCS sinh ra là một PEO sau khi chọn đúng chiều thứ tự.

6.2 LexBFS

LexBFS cũng chọn từng đỉnh một, nhưng nhãn của mỗi đỉnh là một dãy số thay vì một trọng số đơn. Ban đầu mọi nhãn rỗng. Ở bước i , chọn một đỉnh chưa chọn có nhãn lớn nhất theo thứ tự từ điển, rồi thêm chỉ số i vào nhãn của mọi láng giềng chưa chọn của nó. Slide gốc minh họa một lần chạy LexBFS cho thứ tự

$$A, D, C, B, E, F, G, H, J, K, I, L.$$

Để đạt $O(n + m)$, các đỉnh chưa chọn được chia thành các **xô** theo nhãn. Một xô chứa các đỉnh có cùng nhãn; danh sách xô được giữ theo thứ tự giảm dần của nhãn.

1. Lấy đỉnh đầu tiên v_i trong xô đầu tiên; đây là một đỉnh có nhãn lớn nhất hiện tại.
2. Xóa v_i khỏi xô của nó; nếu xô rỗng thì xóa xô.
3. Với mỗi láng giềng chưa chọn w của v_i , chuyển w từ xô mang nhãn $L(w)$ sang xô mang nhãn $L(w) \circ i$.
4. Nếu xô mới chưa tồn tại ngay trước xô cũ trong danh sách, tạo xô mới tại vị trí đó.
5. Nếu xô cũ trở nên rỗng, xóa nó.

Ở đây $L(w) \circ i$ là nhãn thu được bằng cách nối thêm i vào nhãn hiện tại.

6.3 Kiểm tra một thứ tự

Sau khi MCS hoặc LexBFS sinh ra một thứ tự $v_1, \dots, v_n$, ta cần kiểm tra nó có phải PEO hay không. Với quy ước tiền nhiệm, với mỗi v_i đặt $p(v_i)$ là tiền nhiệm có chỉ số lớn nhất của v_i nếu tồn tại. Khi đó thứ tự là PEO nếu và chỉ nếu với mọi tiền nhiệm u khác $p(v_i)$ của v_i , ta có cạnh $(u, p(v_i))$.

Điều kiện này kiểm tra rằng toàn bộ $\text{Pred}(v_i)$ là clique, nhưng chỉ cần so sánh với một đỉnh đại diện $p(v_i)$, vì nếu thứ tự là PEO thì các kiểm tra tương tự ở các bước trước đã bảo đảm phần còn lại. Có thể cài đặt kiểm tra trong $O(n + m)$ bằng danh sách kề và mảng đánh dấu tạm thời.

Ứng dụng trực tiếp là bài *Fishing Net* trên ZOJ: cho một đồ thị, hãy quyết định nó có phải đồ thị dây cung hay không. Ta chạy MCS hoặc LexBFS, rồi kiểm tra PEO.

7 Nhận biết đồ thị khoảng

7.1 Ba đặc trưng tương đương

Với một đồ thị G , các mệnh đề sau là tương đương:

1. G là đồ thị khoảng.
2. G là đồ thị dây cung và G là đồ thị co-comparability, nghĩa là đồ thị bù $\overline{G}$ là một đồ thị comparability.
3. Các clique cực đại của G có thể được đánh số liên tiếp $C_1, C_2, \dots, C_k$ sao cho với mọi đỉnh v , tập chỉ số

$$\{j \mid v \in C_j\}$$

là một đoạn các số nguyên liên tiếp.

Một đồ thị vô hướng được gọi là **comparability graph** nếu các cạnh của nó có thể được định hướng sao cho hướng thu được không có chu trình và có tính bắc cầu: nếu có cung $x \rightarrow y$ và $y \rightarrow z$ thì cũng có cung $x \rightarrow z$.

Hàm ý (3) $\Rightarrow$ (1) rất trực tiếp. Nếu các clique cực đại đã được xếp theo thứ tự liên tiếp, ta gán cho mỗi đỉnh v khoảng

$$I(v) = [\min\{i \mid v \in C_i\}, \max\{i \mid v \in C_i\}].$$

Hai đỉnh kề nhau khi và chỉ khi chúng cùng nằm trong một clique cực đại nào đó, tức hai khoảng tương ứng giao nhau.

Hàm ý (1) $\Rightarrow$ (2) đến từ việc đồ thị khoảng là đồ thị dây cung, còn đồ thị bù của nó có thể định hướng theo thứ tự tương đối của các khoảng không giao nhau: nếu I_x hoàn toàn nằm bên trái I_y , định hướng cạnh bù thành $x \rightarrow y$. Hướng này là không chu trình và bắc cầu.

Với hàm ý (2) $\Rightarrow$ (3), slide gốc xây dựng một đồ thị có hướng H trên tập các clique cực đại. Lấy $\overline{G}$ sau khi đã định hướng bắc cầu và không chu trình thành G' . Đặt $V(H)$ là tập các clique cực đại của G ; với hai clique C_1, C_2 , thêm cung $C_1 \rightarrow C_2$ nếu tồn tại $x \in C_1, y \in C_2$ sao cho $x \rightarrow y$ trong G' . Các định lý trên slide khẳng định: H có tính bắc cầu, H không có chu trình, và một thứ tự topo $C_1, C_2, \dots, C_k$ của H chính là thứ tự clique thỏa điều kiện liên tiếp.

7.2 Liệt kê clique cực đại trong đồ thị dây cung

Giả sử G là đồ thị dây cung và ta đã có một PEO $v_1, \dots, v_n$ theo quy ước tiền nhiệm. Khi đó mỗi clique cực đại của G có dạng

$$\{v_i\} \cup \text{Pred}(v_i)$$

với một chỉ số i nào đó.

Không phải mọi tập ở dạng trên đều là clique cực đại. Nó là clique cực đại khi và chỉ khi không có đỉnh xuất hiện sau nào chứa toàn bộ tập này trong clique của nó. Cụ thể, với mọi hậu nhiệm v_j của v_i kề với v_i , phải tồn tại một tiền nhiệm của v_i không phải là tiền nhiệm của v_j . Nếu ngược lại mọi tiền nhiệm của v_i cũng kề với v_j , thì clique của v_i bị chứa trong clique lớn hơn ứng với v_j .

Nhờ vậy, sau khi nhận biết chordal và có PEO, ta có thể liệt kê các clique cực đại để chuyển sang bước kiểm tra tính liên tiếp.

7.3 Tính chất các số 1 liên tiếp

Tính chất các số 1 liên tiếp (consecutive ones property, C1P hoặc COP) của một ma trận nhị phân nói rằng có thể hoán vị các cột sao cho trong mỗi hàng, các phần tử bằng 1 xuất hiện thành một khối liên tiếp.

Với bài toán nhận biết đồ thị khoảng, lập ma trận A có hàng là các đỉnh và cột là các clique cực đại:

$$a_{ij} = 1 \iff v_i \in C_j.$$

Khi đó đồ thị dây cung G là đồ thị khoảng khi và chỉ khi ma trận này có C1P. Slide gốc nhắc tới POJ 2790 như một bài kiểm tra trực tiếp xem một ma trận có tính chất C1P hay không.

Ví dụ, nếu $N = 4$ và ràng buộc đầu tiên là tập $S_1 = \{2, 3\}$ phải đứng liên tiếp, các hoán vị hợp lệ của 1, 2, 3, 4 là những hoán vị trong đó 2 và 3 đứng cạnh nhau. Nếu thêm ràng buộc $S_2 = \{3, 4\}$, tập hoán vị còn lại bị thu hẹp hơn nữa, chẳng hạn các thứ tự như

$$\langle 1, 2, 3, 4 \rangle, \quad \langle 1, 4, 3, 2 \rangle, \quad \langle 2, 3, 4, 1 \rangle, \quad \langle 4, 3, 2, 1 \rangle$$

đều thỏa cả hai ràng buộc. Đây chính là kiểu thông tin mà PQ-tree lưu trữ một cách nén.

7.4 PQ-tree

PQ-tree là một cây biểu diễn tập tất cả các hoán vị cột còn có thể thỏa các ràng buộc C1P đã xử lý. Lá của cây là các cột. Nút trong có hai loại:

- **P-node**: các con của nó có thể được hoán vị tùy ý;
- **Q-node**: thứ tự các con của nó cố định tới phép đảo ngược.

Khi thêm một ràng buộc mới S yêu cầu các lá thuộc S phải liên tiếp, thuật toán đánh dấu các nút theo quan hệ với S :

- **full**: toàn bộ lá trong cây con thuộc S ;
- **empty**: không có lá nào trong cây con thuộc S ;
- **partial**: chỉ một phần lá trong cây con thuộc S .

Nút thấp nhất bao trùm toàn bộ phần liên quan tới S được gọi là **pertinent root**. Từ nút này, cây được rút gọn bằng một số mẫu thay thế cục bộ.

Các slide gốc liệt kê các mẫu chuẩn:

- L1: lá thuộc S được đánh dấu full;
- P1: P-node có toàn bộ con full thì trở thành full;
- P2: P-node là pertinent root với các con full và empty; gom các con full dưới một P-node mới nếu có nhiều hơn một con full;
- P3: P-node không phải pertinent root với con full và empty; biến nó thành Q-node partial, gom nhóm full và nhóm empty bằng các P-node phụ khi cần;
- P4, P5, P6: các trường hợp P-node có một hoặc hai con partial, cộng thêm nhóm full và empty, tùy nó có phải pertinent root hay không;
- Q1: Q-node có toàn bộ con full thì trở thành full;
- Q2: Q-node có không quá một con partial, một khối full liên tiếp, và phần còn lại empty;
- Q3: Q-node có hai con partial ở hai đầu của một khối full liên tiếp, phần còn lại empty.

Nếu một ràng buộc không khớp được với mẫu hợp lệ nào, ma trận không có C1P và đồ thị không phải đồ thị khoảng. Nếu xử lý hết các hàng, bất kỳ thứ tự lá nào do PQ-tree còn lại biểu diễn đều cho một thứ tự các clique cực đại thỏa điều kiện liên tiếp, từ đó dựng được biểu diễn khoảng.

8 Đồ thị hoàn hảo

Một đồ thị G được gọi là **đồ thị hoàn hảo** nếu với mọi đồ thị con cảm sinh H của G , ta đều có

$$\omega(H) = \chi(H).$$

Ở đây $\omega(H)$ là kích thước clique lớn nhất của H , còn $\chi(H)$ là số màu nhỏ nhất của H .

Đồ thị khoảng là hoàn hảo vì mọi đồ thị con cảm sinh của đồ thị khoảng vẫn là đồ thị khoảng, và trên đồ thị khoảng ta có $\omega = \chi$. Rộng hơn, đồ thị dây cung cũng là đồ thị hoàn hảo: mọi đồ thị con cảm sinh của nó vẫn dây cung, có PEO, và tô màu tham lam theo PEO đạt đúng kích thước clique lớn nhất.

Định lý đồ thị hoàn hảo mạnh. Một đồ thị là hoàn hảo khi và chỉ khi nó không chứa chu trình lẻ cảm sinh độ dài ít nhất 5 và cũng không chứa phần bù của một chu trình như vậy dưới dạng đồ thị con cảm sinh. Các chu trình lẻ cảm sinh độ dài ít nhất 5 thường được gọi là **odd holes**; phần bù của chúng là **odd antiholes**.

Định lý này đặt đồ thị khoảng và đồ thị dây cung vào bức tranh rộng hơn: chúng là các lớp con có cấu trúc mạnh của lớp đồ thị hoàn hảo, nơi nhiều bài toán tối ưu hóa trên đồ thị có thể được giải hiệu quả nhờ các đặc trưng thứ tự và clique.